

Exercício de Fixação — TADs e Orientação a Objetos

Página / Slide 22

Aluno: Rodrigo Del Padre

RA: 24042092-2

Curso: Engenharia de Software (6º Semestre)

Turma: B

Disciplina: Paradigmas de Programação

Referência: R. Sebesta, 11ª ed., Caps. 11 e 12

Trecho 3 — Java

Código analisado:

```
class A { String nome = "A"; String getNome() { return nome; } }  
class B extends A { String nome = "B"; String getNome() { return nome; } }  
A x = new B();  
System.out.println(x.nome + " " + x.getNome());
```

(a) Qual é a saída?

A B

(b) A decisão foi tomada na compilação ou na execução?

Campo `x.nome`: COMPILAÇÃO | Método `x.getNome()`: EXECUÇÃO

Conceito Teórico: Ocultação de Campos (*Field Shadowing / Hiding*) vs. Sobrescrita Polimórfica de Métodos (*Method Overriding*) (Sebesta, 11ª ed., §12.2.2 e §12.3.5, p. 489–492 e 497; Slides 14, 15 e 17).

- **Campos NÃO são polimórficos:** Em Java, atributos de instância não participam de tabela virtual de métodos. Eles são estaticamente resolvidos pelo compilador com base no tipo estático da variável referenciadora. Como `x` é do tipo estático `A`, o compilador insere `getField A.nome`, acessando "A" na compilação.
- **Métodos SÃO polimórficos:** A chamada `x.getNome()` é despachada dinamicamente via *vtable* pela JVM em tempo de execução, chamando a implementação do objeto real (`B.getNome()`), que retorna o campo local "B".
- **Resultado:** A combinação direta gera "A B".

Trecho 4 — Python

Código analisado:

```
class Contador:  
    total = 0  
    def __init__(self):  
        Contador.total += 1  
        self.id = Contador.total  
  
a = Contador(); b = Contador()  
print(a.id, b.id, a.total)
```

(a) Qual é a saída?

1 2 2

(b) A decisão foi tomada na compilação ou na execução?**Decisão tomada na EXECUÇÃO (Runtime)**

Conceito Teórico: Atributos de Classe (*Class Variables*) vs. Atributos de Instância (*Instance Variables*) e Resolução Dinâmica de Identificadores via `__dict__` (Sebesta, 11ª ed., §12.3.1 e §12.4.6, p. 493–494 e 523–525; Slides 16 e 18).

- **Variável de Classe (`Contador.total`):** Compartilhada universalmente por todas as instâncias da classe `Contador` no namespace da classe.
- **Variável de Instância (`self.id`):** Atribuída ao dicionário interno individual de cada instância no método `__init__`.
- **Sequência de Execução:**
 - Ao instanciar `a = Contador()`: `Contador.total` passa a ser 1; `a.id` é vinculado ao valor 1.
 - Ao instanciar `b = Contador()`: `Contador.total` passa a ser 2; `b.id` é vinculado ao valor 2.
- **Resolução de `a.total`:** O interpretador faz o **lookup** de `'total'` no objeto `a`. Não encontrando no dicionário de instância, ele ascende para o dicionário da classe `Contador`, encontrando o valor atual 2. Em Python, toda amarração e resolução ocorre dinamicamente na execução.